Project 1 Report

Jayden Cozort

Part 1: Build and Understand

Data Processing

For data the *GitHub Code Snippets - Development sample*[1] was used for training. To process the data, the data's sqlite was first downloaded and the snippet tag(the code of each github link) were all combine into a single line in correspondent with the repo_file_name and github_repo_url so that the whole code file were all together, then the combined snippets dataset was filtered to only contain code snippets from the C programming language(most dense language in dataset). After filtering the data to only contain C code the dataset was then split into a 90% training, 10% validation data set and tiktoken's gpt4 encoder was used after modifications to reduce the vocabulary size from 100,261 down to 8,192 tokens. After the tokenization of the data was complete, the tokenized dataset was then flattened and put into a train.bin and val.bin respectively to the split dataset. These files came out to a size of 67,4 MB with ~35 M tokens for the train.bin and 7.5 MB with ~4 M tokens for the val.bin file.

Training and Configuration

Once the data was tokenized, the model was then created using a slightly modified version of nanoGPT, modifications were mainly made for the ablation study and dealing with bf16 incompatibility. The models configuration consisted of:

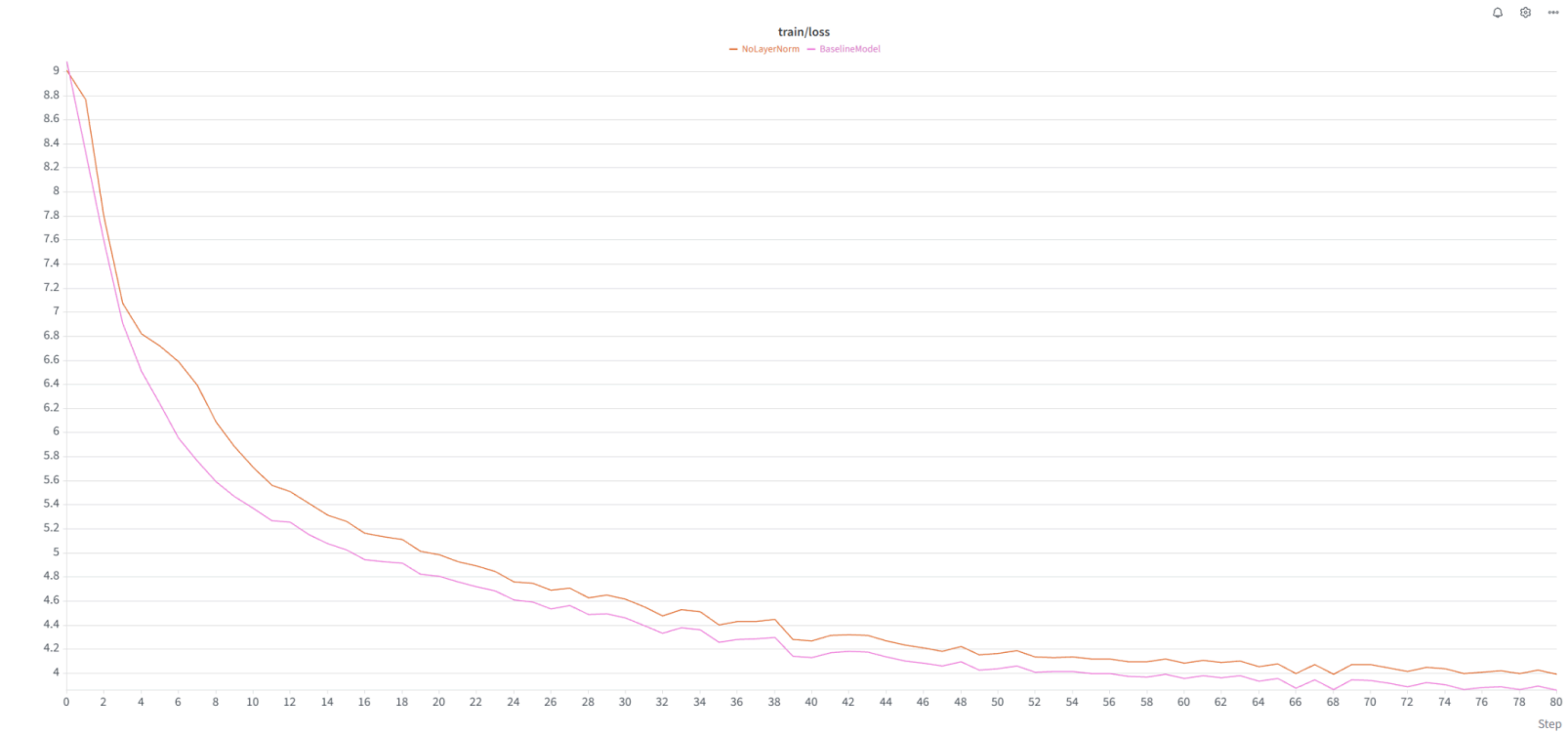
- 6 Layers / 6 Heads / 384 Embedding Size
- 512 token Context length
- BPE tokenizer with 8,192 vocab size
- 35M training tokens
- float16 precision(bf16 not supported)

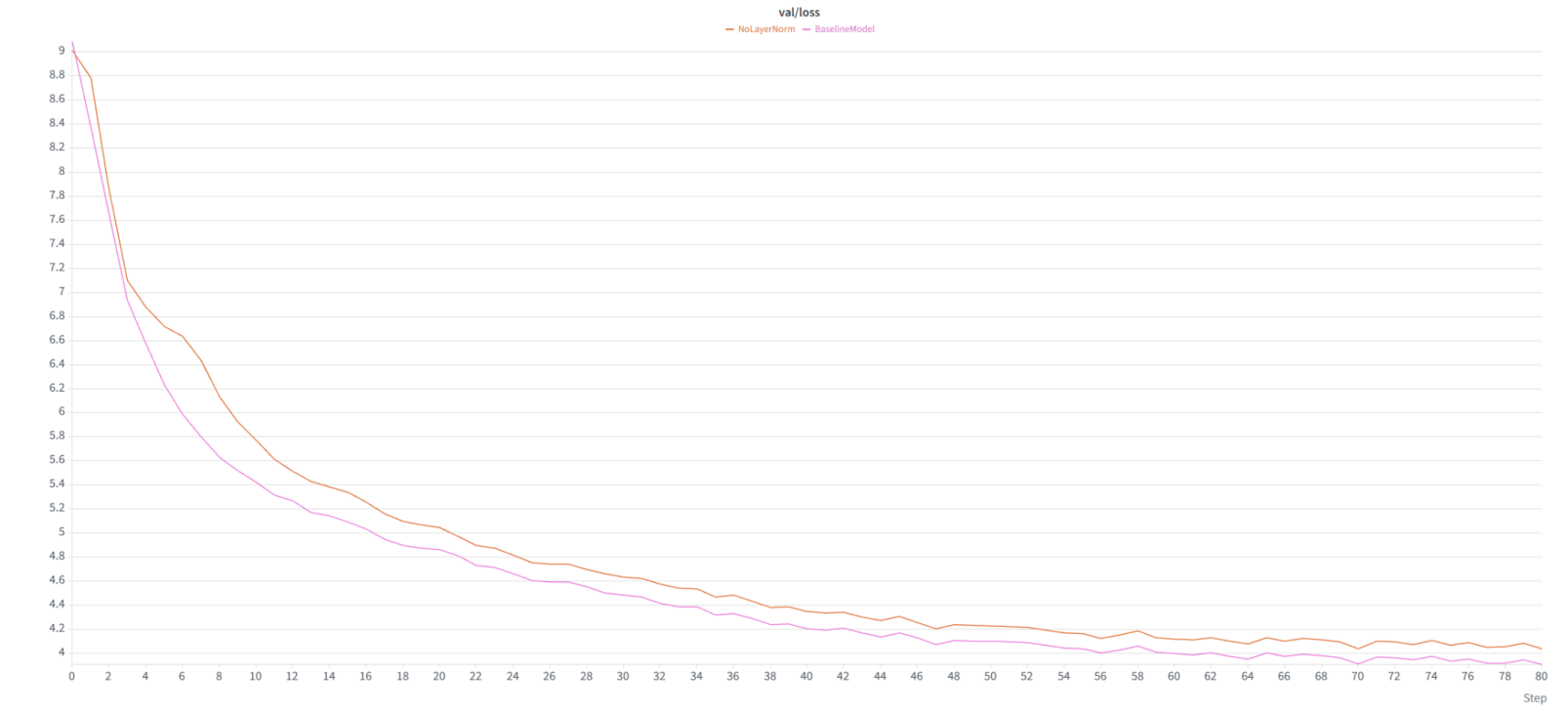
The model was then trained for 800 iterations with sampling for weight, loss, and checkpointing happening every 10 iterations. The run was then saved to wandb for training and validation loss.

Ablation Study

For the ablation study, layer normalization was removed from the ablation model with all other settings staying the exact same for parity. The predictions for removing layer normalization were that the training would become less stable in addition to answers given by the model would be worse. After training was completed, these predictions were then identified to be correct with the training and validation loss, although saw a faster initial decrease in the first epochs of training. It started to plateau at a training loss and validation loss of ~4 and 4.1, whilst the baseline model's was slightly lower at ~3.9

for both as seen in the provide images folder in GitHub. Additionally both were run with the prompt “int n = 10\n\nif(isPrime(n)). The Baseline model proceeds to fill in the if to a degree of possible output such as “return 0”, although was generally followed by recurring token chaining. While the model with no layer normalization failed to generate anything after the prompt under the same settings. This shows that both predictions are correct to a degree .As the training had become more consistent and the non layer normalization model didn’t even generate an answer.





Reflection

The model seems to be able to create code to a degree but fails to understand what type of code one wants or grasp relations that are less common. Additionally it gets stuck often with generating the same token over and over again, even with a high top-k. Because of this, the model is very limited in what it can do and would most likely never create the code the user is looking for without some editing being required.

Part 2: Use the Model on a Real SE Task

For testing the models capabilities 20 segments of simple programs were taken from the geeksforgeeks website[2] with a focus on code completion for those programs. This was done using a single text file named Phase2Inputs.txt with each prompt separated by a '~' character. This was then run through sample.py and outputs were then saved to phase 2 outputs.

Due to the size of the model the concept of accept with editing is more loosely defined. For this instance, if the code generated completes some action that the coder would've had to take, then it can be accepted. If there is no relation to the solution then it is declined. Taking into account the 20 prompts given to the model:

- 0 were accepted as is which is predictable due to the models size
- 9 outputs could be accepted after editing
- 11 could not be used in any way.

For the best case output came from the file renaming test, in the output the model had successfully renamed the file without any syntax error, but editing was still needed due to an added if statement and it renaming the file to name instead of newfile. For a near miss case, this came from the greatest common denominator test, where it filled in the return statement and had closed off the function, the issue with almost completing what it was meant to do is that it returned 0 instead of result. The worst case comes from the atoi conversion program, where it not only doesn't even reference any code, but prints multiple lines of comments with no meaning instead.

Overall the model itself was not very useful in a majority of cases that involved more than one line of code. If the prompt involved filling in a small portion given the code before it, it could fill in some syntax related things but would fail to do much in terms of completing what is expected given the code and comments.

Part 3: Evaluate & Break

For evaluating the weaknesses of the model, 20 prompts were created and stored in Phase3Inputs.txt. For testing, the same scheme of using a delimiter and storing the output in an output text file were done. The probes focus on the model's behaviour in ignoring information present in the prompt, its inability to understand comments if no code follows, its recursive prediction behaviour that leads to unparsable output, and its

calling of a function that doesn't exist rather rare and depends on variable names. 10 failure instances were then recorded along with the input, relevant output, category of failure, severity, and an explanation of the failure. These failures were written down and exported to a csv available in the GitHub repository. The failures are generally distributed evenly when given the probes were focused on certain areas; but one failure area stood out as one of the most prominent. This area was the syntax area, or unparsable output. Many of the outputs resulted in repeated token predictions where the token would end up predicting itself or a similar token that would end up predicting back.

Because of this, when it comes to the failures of the model, multiple areas could be the cause. These possibilities range from overfitting in the model to certain tokens being weighed heavily more than other tokens, in addition to the model just not having enough data in the data set to work with. These improvements would have to be made to the model since a little over half of the failures would be findable in a regular test without mediocre prompts designed to elevate these.

Part 4: Improve & Re-evaluate

For making improvements to the model so that it can work better on the probe and in general. Multiple areas can be touched with a majority going into the data area and training the model. For the data area, the most straightforward way to deal with it is through downloading the 60 gb version of the code snippets data base that was used. Given that it was only 5% of the original, this would help balance the data and avoid overfitting. If this change were to be made, the vocab size of the tokenizer should also be increased to compensate. Another option would be to add l1 or l2 regularization to the model in addition to dropout, this helps with the overfitting issue that leads to the repetitive token predictions, which is creating the bulk of the unparsable outputs. Another solution would be through fine tuning of the model, although this would not be a concrete solution, because it would still see the issues if it were to se a prompt with functions and code outside of what it was finetuned on.

For part 4 adding dropout to the model was the choice, this was done by changing the config file to add a dropout value of 0.15 for the model. This resulted in the training a validation loss increasing with a slight reduction in the computation time. As for testing on the probe. Due to time constraints I am not able to as a result I can not say whether the change worked for dealing with the unparseable output and whether the cost to accuracy in general prompting is worth it.

Discussion

This project has helped me in my understanding of gpt and tokenization. Before the project and the class, a majority of transformers concepts were only sparsely taught and tokenization alluded me. The main learning segment was the building of the model and how tweaks to the configuration can result in major changes int output and training time if they weren't small. It especially helped with understanding tokenizers, far more than before the project given that designing a custom tokenizer was heavily needed given the compute constraints of Collab. This project also changes my view in the effort that goes into these AI assistants given the required effort to process the data in a way where it isn’t unbalanced, the tokenization to the model can understand multiple different syntax types(code, literature, differing language structures), and the importance in instructional tuning and how it removes a major headache in prompt engineering.

Appendix

For using AI gemini was the only source due to its relatively easy use given its connection to the Google Collab environment. A majority of the usage involved speeding up code that I would have already written, with the exception of developing a tokenizer that had kept gpt4's tokenizer's capabilities with a reduce vocab size for far faster training. When Gemini had generated code I focused on looking through it finding what it may have done wrong, and if there was something I didn't know I would both ask Gemini and look into it online. A log of the prompts for code generation are below and what they were used for:

Prompt: "Please group snippets together by their repo_file_name and github_repo_url in the dataset using group_concat from the SQLite library"
After Action: minor editing to query needed
Usage: for combining snippets together, found that this may have been damaging the models capabilities so I had backtracked on it.

Prompt: "Please filter out snippets belonging to all languages except for: python, C, and C++"
After Action: editing to filtered language to house only python needed to reduce training time even further.
Usage: already knew everything gemini did, mainly did it for a time save

"help build the custom encoder at the start to be a vocabulary of 8192 with special tokens that help tokenize python code similar to the cl100_base encoder"
After Action: asked gemini for code explanation, slight editing to map special tokens within 8,192 vocab size.

Usage: Initially had issues understanding the build process but after looking into it found a slight bug that caused issues then fixed it.

Still not used to using AI as a helper, but this project along with Gemini's ease of access helped in moving towards the direction of it being an educational option given that you check over what it says and does.

References

[1] zomglings, “GitHub Code Snippets - Development Sample,” kaggledatasets, Mar. 07, 2021. https://www.kaggle.com/datasets/simiotic/github-code-snippets-development-sample.

[2]GeeksforGeeks, “C Programs,” GeeksforGeeks, Oct. 27, 2022. https://www.geeksforgeeks.org/c/c-programming-examples/.